

Modelowanie podsystemów

dr hab. Michał Śmiałek, prof. uczelni
dr Kamil Rybiński



**Wydział
Elektryczny**
POLITECHNIKA WARSZAWSKA

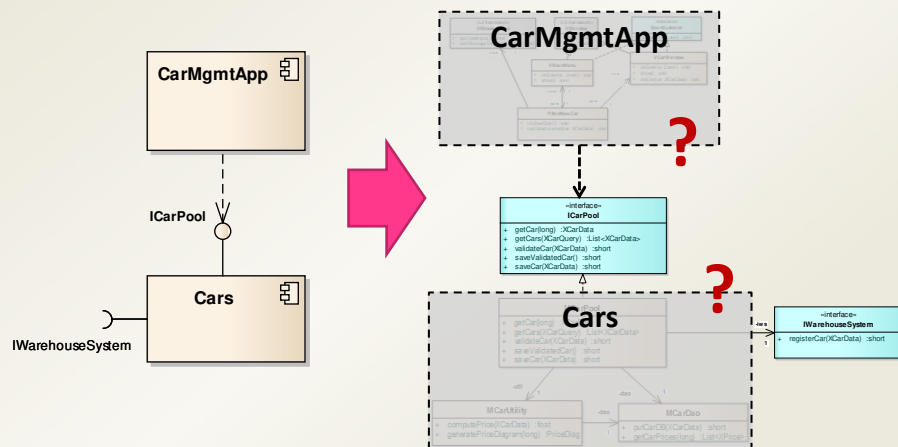
1DI1306: Modelowanie oprogramowania w języku UML

1

Od modelu architektonicznego do modeli podsystemów (przypomnienie)

Diagramy komponentów - widok „z lotu ptaka”

– Komponenty, interfejsy (nazwy), relacje

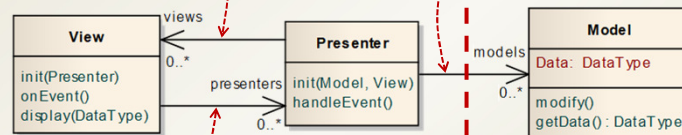


2

Przypomnienie: wzorzec MVP i nawigowalność

Wyświetlanie elementów ekranowych (sterowanie $P \rightarrow V$, przepływ danych $P \rightarrow V$, czasami przepływ danych $V \rightarrow P$ dla okienek modalnych)

Przetwarzanie danych (sterowanie $P \rightarrow M$, przepływ danych $P \rightarrow M$ i/lub $M \rightarrow P$)



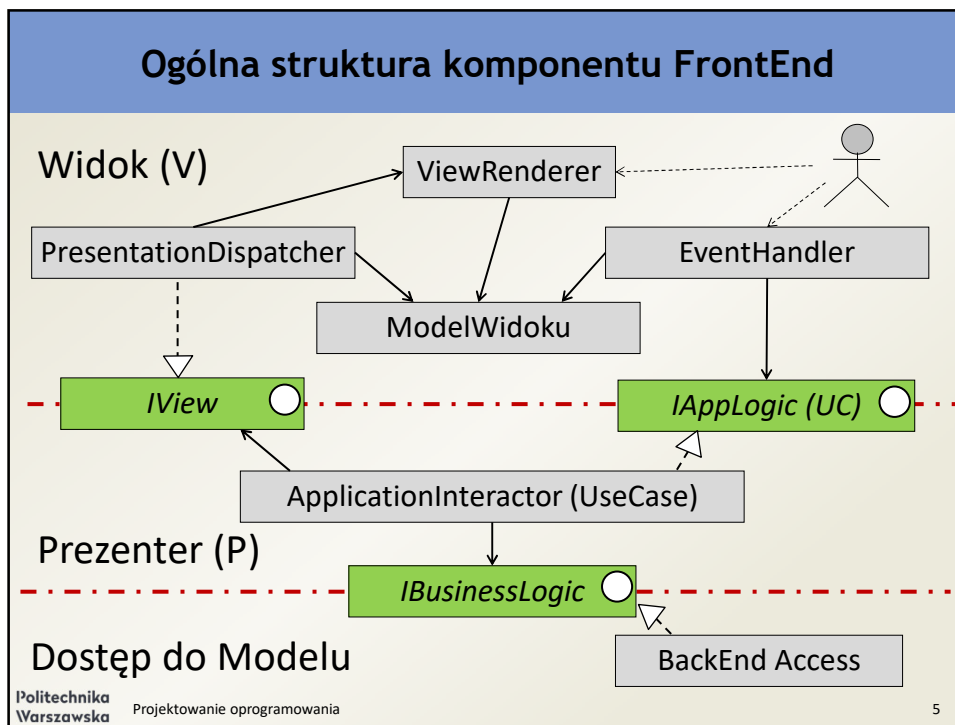
Obsługa zdarzeń (sterowanie $V \rightarrow P$, przepływ danych $V \rightarrow P$)

FrontEnd ! BackEnd

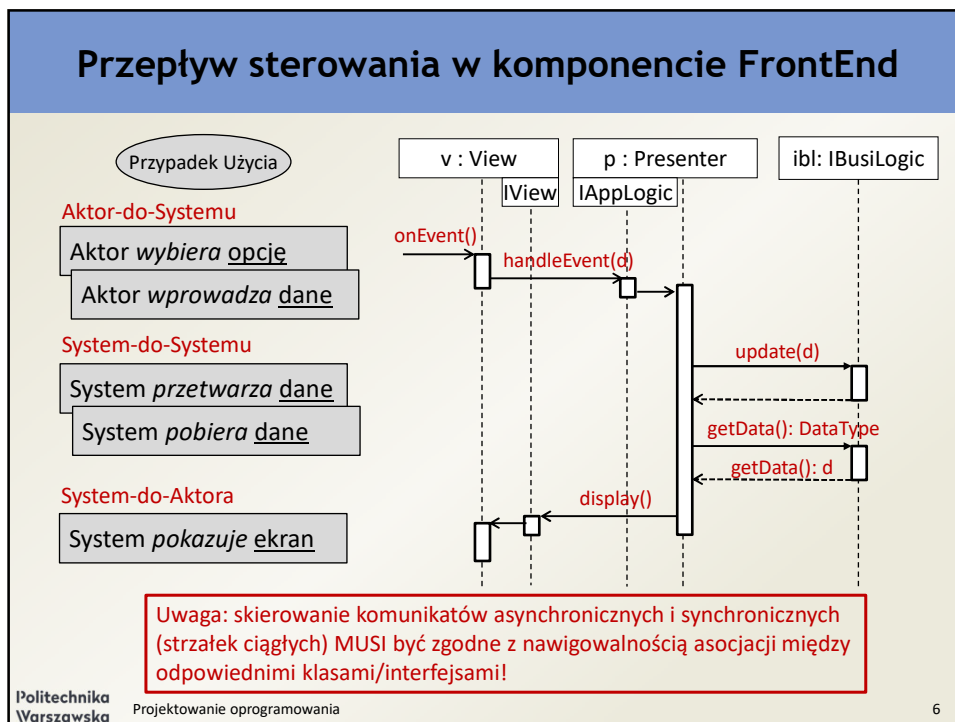
Uwaga: nawigowalność asocjacji (kierunek strzałki) NIE determinuje kierunku przepływu danych!

Projektowanie podsystemów „front-end”

- Co należy zaprojektować?
 - Elementy ekranowe, czyli:
 - Renderowanie okienek, pól, przycisków, kontrolek, ...
 - Logika aplikacji, czyli:
 - Nawigacja między elementami ekranowymi
 - Obsługa zdarzeń, uruchamianie elementów ekranowych
 - Odwołania do logiki dziedzicznej (przetwarzania danych)
- Uwaga: projektowanie silnie zależy od technologii front-end
 - Przeważa JavaScript: React, Vue, Angular
 - JavaScript - bezpośrednie wsparcie przez przeglądarki
 - Inne technologie, np. oparte na Javie: Java FX, Vaadin
 - Java - konieczne tłumaczenie, ew. lokalna maszyna wirtualna Java



5



6

Modelowanie elementów ekranowych

- Założenie: elementy ekranowe implementowane w języku obiektowym
 - Model klas odzwierciedla pełną strukturę okienek z wszystkimi istotnymi elementami składowymi (pola, przyciski, kontrolki, ...)
- Założenie: framework dostarcza odpowiedniej struktury klas „okienkowych”
 - Okienka i inne elementy ekranowe specjalizują (dziedziczą) od klas frameworka
- Nawigacja między okienkami jest sterowana poprzez interfejs
 - Klasa implementująca (realizująca) interfejs odpowiada za tworzenie okienek i wywoływanie procedur ich wyświetlania

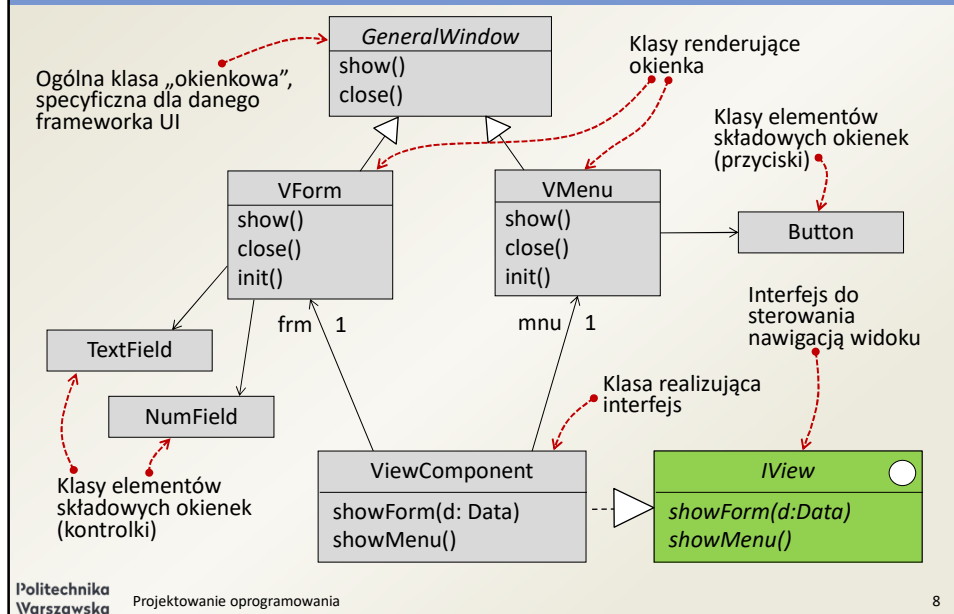
Politechnika
Warszawska

Modelowanie architektury

7

7

Przykładowa struktura warstwy Widoku (V)



Politechnika
Warszawska

Projektowanie oprogramowania

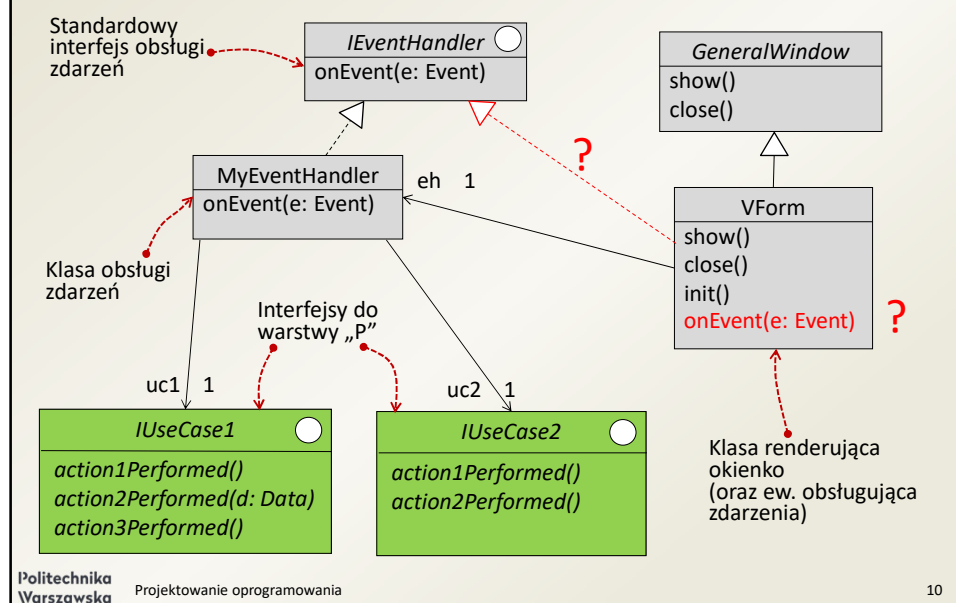
8

8

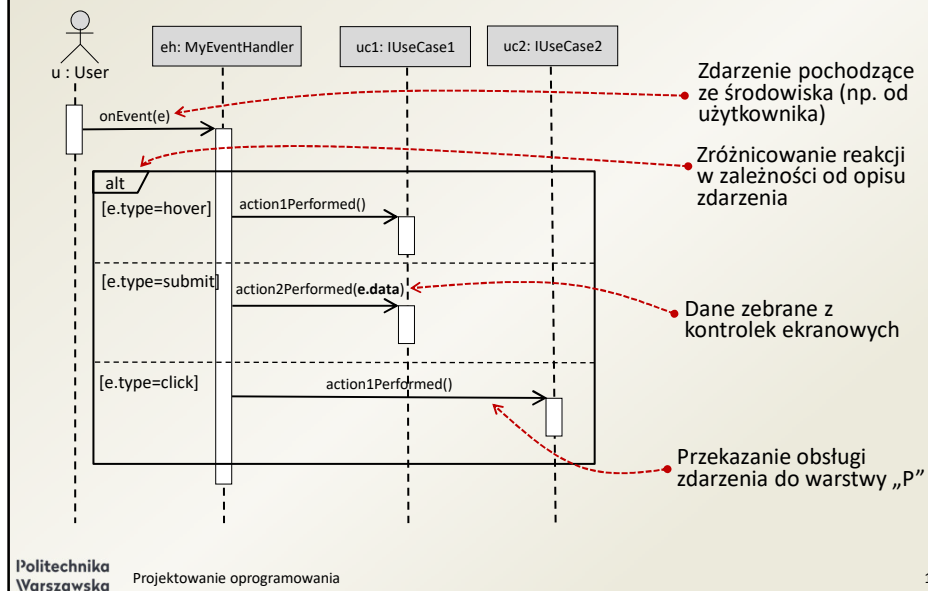
Modelowanie zdarzeń

- Co to jest zdarzenie?
 - Akcja lub wystąpienie, rozpoznawane przez software
 - Przekazywane zazwyczaj asynchronicznie ze środowiska
 - Może być obsłużone przez odpowiedni fragment software'u
- Zdarzenia interfejsu użytkownika
 - Przykładowo: kliknięcie myszą, przesunięcie myszy nad element, zmiana zawartości pola, naciśnięcie klawisza, ...
- Obsługa zdarzeń
 - Wykorzystanie mechanizmów dostarczanych przez framework
 - Określenie procedur obsługi zdarzeń (np. realizacja interfejsu)
 - Zazwyczaj mechanizmy frameworka zbierają dane z kontrolek, ale czasami kod widoku musi te dane z kontrolek „ręcznie” odczytać
 - Zdarzenie oraz dane przekazujemy do obsłużenia przez „P”

Struktura warstwy widoku: obsługa zdarzeń

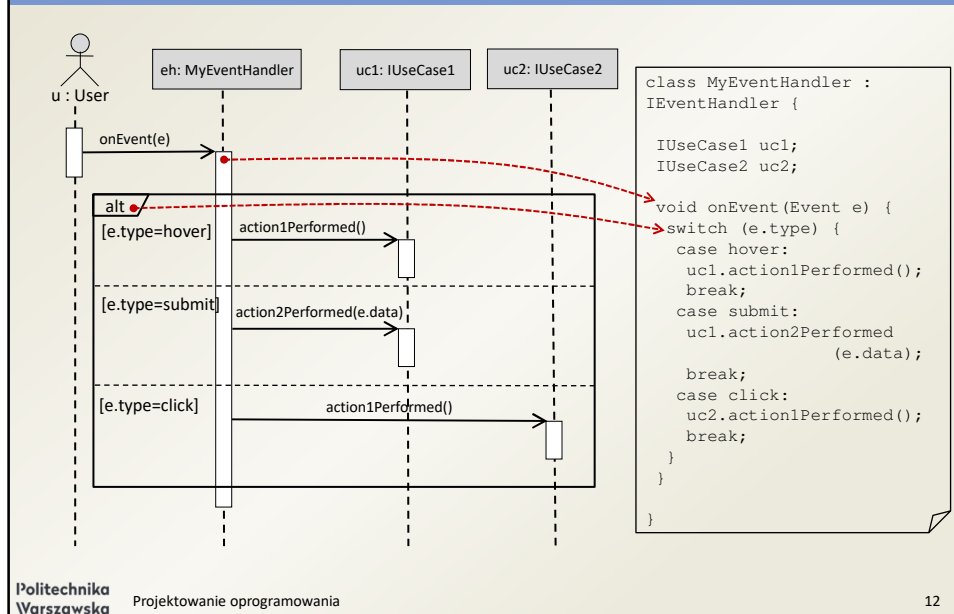


Obsługa zdarzeń: przykładowy przepływ sterowania



11

Obsługa zdarzeń: przykładowy przepływ sterowania, kod



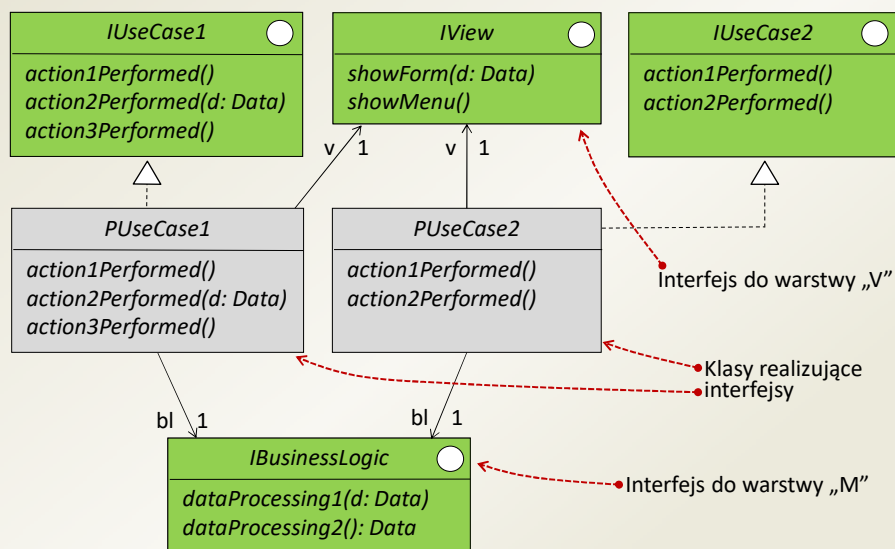
12

Modelowanie logiki aplikacji

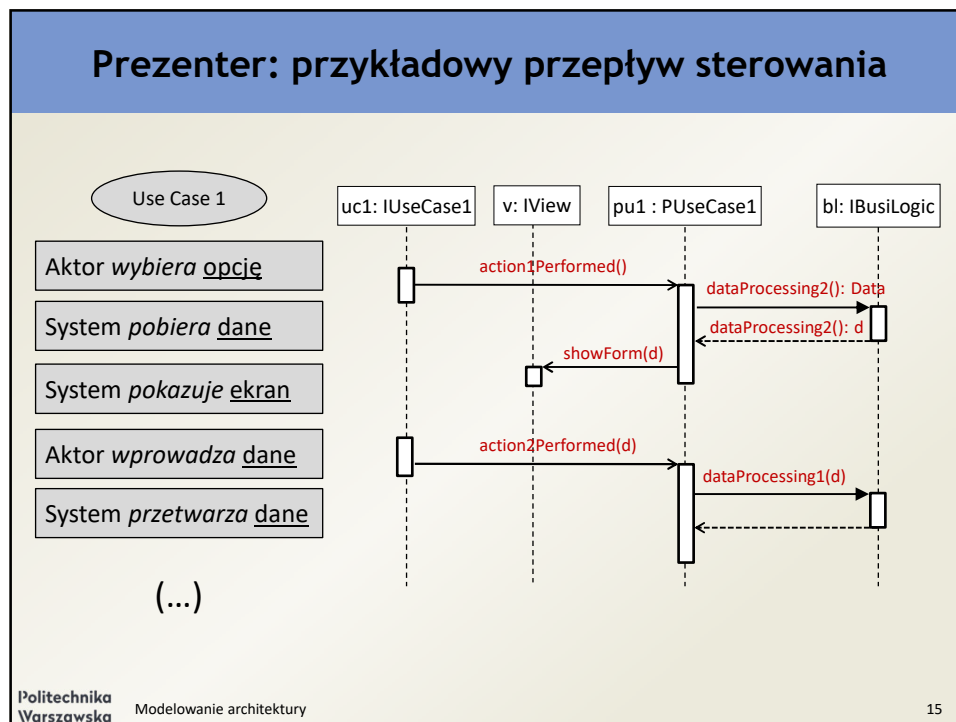
- Logika zachowania systemu
 - Reakcja na decyzje użytkownika
 - Reakcja na wyniki przetwarzania danych
- Wywoływanie operacji nawigacji między okienkami
 - Odwołania do warstwy „V”
- Wywoływanie operacji przetwarzania danych
 - Odwołania o warstwy „M”

13

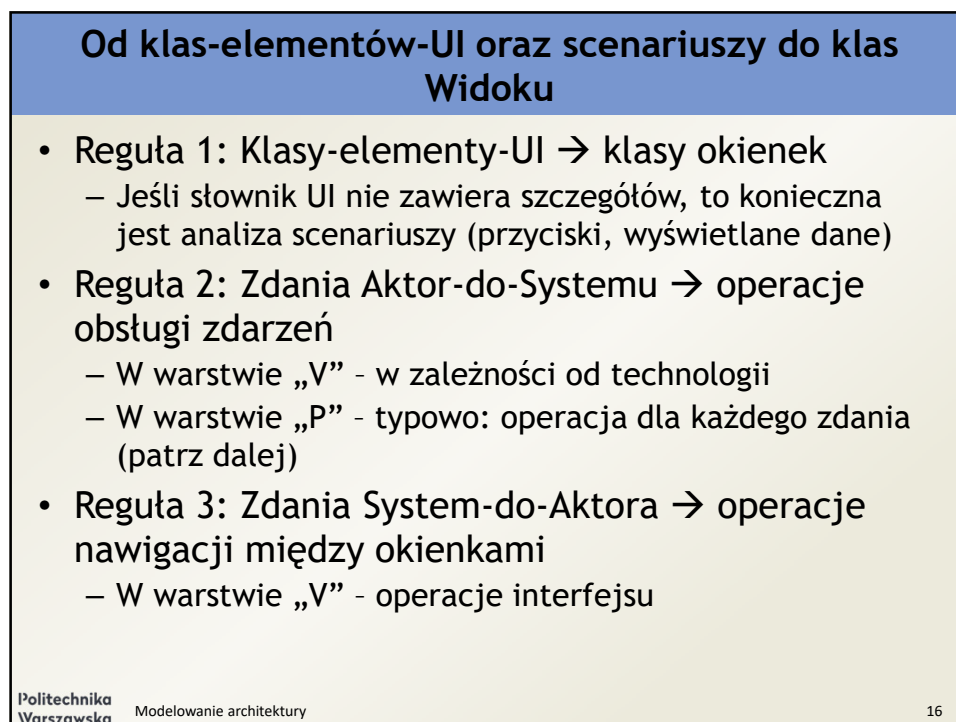
Struktura warstwy prezentera



14



15



16

Od przypadków użycia do klas Prezentera

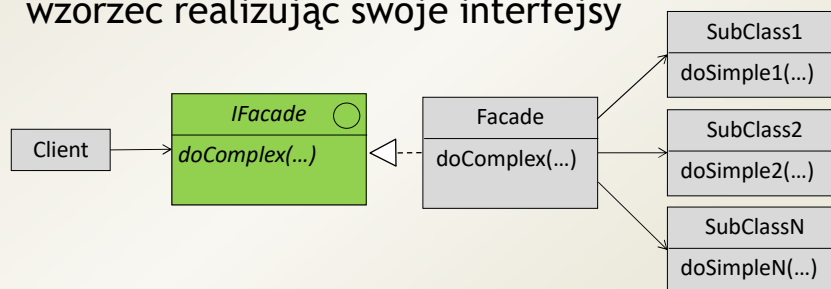
- Reguła 4: Przypadki użycia → klasy Prezentera
 - Dla każdego przypadku użycia klasa realizująca odpowiedni interfejs warstwy „P”
- Reguła 5: Zdania Aktor-do-Systemu → operacje obsługi zdarzeń
 - Patrz reguła 2
- Reguła 6: Zdania System-do-Systemu → relacje do interfejsów warstwy Modelu
 - Umożliwienie wywołania procedur przetwarzania danych zawartych w warstwie „M”

Projektowanie podsystemów „back-end”

- Co należy zaprojektować?
 - Realizacja interfejsów (klasy realizujące)
 - Klasy przechowujące dane (model danych)
 - Klasy przetwarzania danych
 - ~~Struktura bazy danych~~
- Zależność od technologii
 - Realizacja interfejsów - zależy od technologii (REST, GRPC)
 - Struktura bazy danych - zależy od technologii BD (relacyjna, nierelacyjna) i/lub technologii mapowania obiektowo-relacyjnego (np. Hibernate)
 - Pozostałe klasy to zazwyczaj POCO/POJO (Plain-Old-C#/Java-Object)

Fasada: wzorzec projektowy

- Ukrywa detale skomplikowanego podsystemu
- Ułatwia zrozumienie skomplikowanego kodu
- Agreguje funkcjonalność wielu klas
 - Zwiększenie poziomu abstrakcji dla klienta
- Warstwa back-end (Model) często wykorzystuje ten wzorzec realizując swoje interfejsy



19

Realizacja przechowywania i przetwarzania danych

- Przykładowe strategie realizacji komponentów warstwy Modelu
- „Aktywny” model danych
 - Dane przechowywane np. w klasach POCO/POJO bazujących na klasach-pojęciach
 - Dane przetwarzane przez klasy POCO/POJO
 - Konieczne tłumaczenie DTO $\leftrightarrow$ klasy POCO/POJO
- Obiekty transferu danych (DTO) + klasy usługowe
 - Dane przechowywane jako DTO
 - Dane przetwarzane przez klasy usługowe

20

„Aktywny” model danych

- Realizacja złożonego przetwarzania danych
 - Przetwarzanie danych wykonywane na „sieci” obiektów zgodnych z modelem danej dziedziny (paradygmat obiektowy)

